

Controlled LLM-Assisted SAST Triage

Abstract

Static application security testing (SAST) tools can identify vulnerable code patterns, but security teams often struggle with duplicate findings, severity noise, and false-positive or hardening-only results. This project presents a hybrid SAST triage workflow that keeps the scanner as the source of truth while using deterministic clustering and a local large language model (LLM) to improve finding readability and remediation guidance.

The prototype scans small Python applications with Semgrep, groups related findings without dropping scanner check IDs, applies deterministic rules for high-confidence vulnerability classes, and uses a local Ollama model to generate impact and minimal safe-fix explanations. The design intentionally limits LLM authority: the LLM does not invent findings, remove scanner evidence, or decide evidence integrity. Validation checks ensure that every Semgrep check ID remains represented in the final triage output.

An initial evaluation across five targets covers command injection, eval-based code injection, SQL injection, subprocess hardening, and eval hardening cases. The prototype reduced nine raw Semgrep findings into five triage clusters, achieving a 1.80x compression ratio while preserving all scanner check IDs. The initial dataset shows 1.00 recall, 1.00 severity accuracy, and 1.00 false-positive triage accuracy, with 0 missing and 0 extra check IDs.

These early results suggest that a controlled hybrid approach can make SAST output more actionable without treating the LLM as an independent security authority. The current work is an initial prototype and requires broader validation across larger applications, additional languages, more vulnerability classes, and multiple local LLMs.

Introduction

Static application security testing (SAST) is an important part of application security programs because it can identify vulnerable code patterns early in the software development lifecycle. However, the value of SAST depends not only on detection, but also on how quickly developers and security engineers can understand, prioritize, and remediate the findings.

In practice, SAST output can be noisy. A single vulnerable code pattern may produce multiple related findings. Some findings are confirmed vulnerabilities, while others are hardening-only or false-positive cases. This creates triage friction, especially for teams that must review large numbers of findings across multiple applications.

Recent advances in large language models (LLMs) have created new opportunities to summarize, explain, and prioritize security findings. However, using an LLM directly as a security decision engine creates risk. An LLM can omit details, overstate confidence, or generate explanations that are not fully supported by the scanner evidence.

This project takes a controlled hybrid approach. The SAST scanner remains the source of truth. Deterministic logic clusters related findings, preserves check IDs, and applies high-confidence triage rules. A local LLM is then used to enrich each cluster with impact and remediation guidance. This design aims to make SAST output more actionable without giving the LLM authority to invent, drop, or change scanner evidence.

Contributions

This project makes three initial contributions:

- A hybrid SAST triage pipeline that combines Semgrep, deterministic clustering, and local LLM enrichment.
- A validation design that preserves scanner check IDs and flags missing or extra evidence.
- An initial evaluation across five Python targets covering command injection, code injection, SQL injection, and false-positive or hardening-only cases.

Scope

The current work is an early prototype. It is not intended to replace SAST tools or manual security review. Instead, it explores whether a controlled LLM-assisted layer can reduce triage friction while preserving scanner evidence integrity.

Background and Motivation

Static application security testing (SAST) is widely used to identify vulnerable code patterns before software reaches production. SAST tools are valuable because they operate early in the development lifecycle, integrate into developer workflows, and provide concrete evidence such as rule identifiers, file paths, line numbers, and matched code. This evidence makes SAST useful for both developer remediation and security review.

However, SAST output is not always easy to act on. A single vulnerable code path can trigger multiple related rules. For example, a user-controlled input that reaches a dangerous subprocess call may produce one finding for command injection, another for shell usage, and another for dangerous subprocess invocation. These findings are related, but they may appear as separate issues in the scanner output. Without clustering, reviewers may spend time reading multiple findings that describe the same underlying risk.

Severity is also difficult to interpret from raw SAST output alone. A rule may correctly identify a dangerous API such as ``eval()`` or ``subprocess(..., shell=True)``, but exploitability depends on whether untrusted input reaches that operation. Some findings represent confirmed vulnerability paths, while others are better treated as hardening recommendations. This distinction matters because teams need to prioritize exploitable issues without ignoring security-relevant hardening opportunities.

False positives and hardening-only findings create another source of triage friction. A scanner may flag a dangerous API even when the current code uses only internal constants or allowlisted values. These findings are still useful because they identify risky constructs, but they should not be presented with the same urgency as a confirmed injection path. A useful triage system should

separate confirmed vulnerabilities from lower-risk hardening cases while preserving the original scanner evidence.

Large language models (LLMs) appear attractive for this problem because they can summarize technical findings, explain impact, and produce remediation guidance in developer-friendly language. But using an LLM as the primary security decision engine is risky. LLMs may omit details, overstate certainty, invent unsupported evidence, or change the meaning of scanner output. For SAST triage, these risks are especially important because file paths, rule IDs, and line numbers are part of the evidence trail.

This project is motivated by a safer design principle: the scanner remains the source of truth, while the LLM is used only as an enrichment layer. Deterministic logic first groups related findings, preserves Semgrep check IDs, identifies high-confidence patterns, and validates that no scanner evidence is dropped or invented. Only after this evidence-preserving step is the local LLM used to generate impact and minimal safe-fix explanations.

The motivation is not to replace manual review or claim that an LLM can independently determine exploitability. Instead, the goal is to reduce the reading burden for security teams and developers. A reviewer should be able to move from several raw findings to a smaller set of evidence-preserving clusters, each with a clear explanation, severity decision, false-positive assessment, and remediation suggestion.

The initial prototype demonstrates this idea across five small Python targets. Confirmed vulnerability cases include subprocess command injection, eval-based code injection, and SQL injection. Hardening or false-positive cases include subprocess usage without demonstrated user-controlled input and eval usage without demonstrated user-controlled input. These examples are intentionally small so the pipeline, validation logic, and evaluation metrics can be inspected directly.

The broader research question is whether controlled LLM-assisted triage can make SAST findings more usable without weakening evidence integrity. This requires a conservative architecture: deterministic evidence preservation first, local LLM enrichment second, and validation checks before accepting the final triage output.

Related Work

Static Analysis and SAST Triage

Static application security testing (SAST) is a common approach for identifying security-relevant code patterns before deployment. SAST tools are useful because they can provide repeatable evidence such as rule identifiers, source locations, matched code, and data-flow traces. At the same time, static analysis often produces findings that require manual interpretation. OWASP notes that static code analysis tools can report false positives when the tool cannot fully determine whether data is safe as it flows through an application. This creates a practical triage problem: reviewers must distinguish confirmed vulnerabilities from lower-confidence or hardening-only findings.

Semgrep is used in this project as the scanner because it provides structured JSON output, rule identifiers, source locations, and security rule packs that are easy to integrate into an experiment pipeline. This project treats Semgrep output as the source of truth and does not allow the LLM layer to invent or remove scanner evidence.

LLM-Assisted Vulnerability Analysis

Recent work has explored combining large language models (LLMs) with static analysis for vulnerability detection and contextual reasoning. IRIS, for example, proposes a neuro-symbolic approach that uses LLMs with static analysis to support repository-level vulnerability reasoning and taint specification inference. This direction shows that LLMs can help security analysis when they are combined with structured program-analysis signals rather than used as standalone text generators.

The present project is narrower and more conservative. It does not ask the LLM to discover vulnerabilities independently or infer missing scanner evidence. Instead, deterministic logic first clusters findings and preserves check IDs, file paths, and line numbers. The LLM is then used only for impact and remediation explanation after the scanner evidence has already been organized.

LLMs for False-Positive Reduction

Several recent studies focus on using LLMs to reduce SAST false positives. ZeroFalse frames static analyzer output as structured contracts and uses LLMs with contextual evidence and CWE-specific knowledge to improve precision while preserving coverage. QASecClaw proposes a multi-agent approach where a SAST engine first reports candidate vulnerabilities and LLM-based agents then review findings with source-code context to classify likely true positives and false positives. Other recent work studies LLM agents for vulnerability false-positive reduction and reports large reductions in SAST noise on benchmark and real-world datasets.

This project aligns with that general direction but differs in scope and safety posture. The current prototype is not a multi-agent system and does not claim broad benchmark performance. Its contribution is an inspectable, local, evidence-preserving workflow: SAST evidence is kept intact, deterministic rules handle high-confidence cases, and local LLM enrichment is constrained to explanation and remediation text.

Benchmarks and Evaluation

Benchmarking is important because SAST triage systems can appear useful on small examples while failing on larger or more diverse codebases. SASTBench introduces a benchmark for evaluating SAST triage agents using real CVEs as true positives and filtered SAST findings as false-positive candidates. Such benchmarks are useful for future validation of this project because they provide a path beyond small synthetic examples.

The current evaluation is intentionally small. It uses five Python targets covering command injection, eval-based code injection, SQL injection, subprocess hardening, and eval hardening. The goal is to validate the pipeline design, metrics, and evidence-preservation checks before expanding to larger benchmarks and real-world projects.

Positioning of This Work

The related work suggests a growing interest in LLM-assisted vulnerability detection, SAST triage, and false-positive reduction. This project takes a conservative position within that space. It does not position the LLM as the authority over scanner results. Instead, it uses a scanner-first architecture where deterministic clustering, check-ID preservation, and validation occur before LLM-generated explanations are accepted.

This makes the project most relevant to teams that want LLM assistance for developer readability and triage efficiency, but do not want to weaken the evidence trail provided by SAST tools.

References

- [1] OWASP, "Static Code Analysis." OWASP Community Pages.
- [2] Semgrep, "Semgrep documentation" and "semgrep/semgrep: Lightweight static analysis for many languages."
- [3] Z. Li et al., "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities." arXiv, 2024.
- [4] M. Iranmanesh et al., "ZeroFalse: Improving Precision in Static Analysis with LLMs." arXiv, 2025.
- [5] M. R. Ameen, M. T. U. Alam, and A. Islam, "QASecClaw: A Multi-Agent LLM Approach for False Positive Reduction in Static Application Security Testing." arXiv, 2026.
- [6] J. Feiglin et al., "SastBench: A Benchmark for Testing Agentic SAST Triage." arXiv, 2026.
- [7] Y. Xiong et al., "A Comparative Study of LLM Agents in Vulnerability False Positive Reduction." arXiv, 2026.
- [8] "Reducing False Positives in Static Bug Detection with LLMs." arXiv, 2026.

System Design

The prototype is organized as a scanner-first SAST triage pipeline. The design separates evidence collection, deterministic triage, LLM enrichment, and evaluation so that each stage can be inspected independently. This separation is important because the project does not treat the LLM as the source of security truth. The scanner produces evidence, deterministic logic preserves and organizes that evidence, and the LLM improves readability only after validation constraints are applied.

Design Goals

The system has four design goals.

First, scanner evidence must be preserved. Every Semgrep finding contains a check ID, file path, line number, message, severity, and code context. The triage pipeline must not drop or invent check IDs. This is enforced by validating the final LLM-enriched output against the original scanner output.

Second, related findings should be grouped into actionable clusters. A single vulnerable code path may trigger multiple Semgrep rules. Presenting each finding independently can increase reviewer

effort. The pipeline therefore groups related findings by vulnerability pattern, file, and evidence signals.

Third, high-confidence vulnerability patterns should be handled deterministically. For cases such as command injection, eval-based code injection, and SQL injection, the pipeline checks whether user-controlled input reaches a dangerous operation. When that condition is present, the cluster is marked as a likely true positive. When only a dangerous API is present without demonstrated user-controlled input, the cluster can be treated as hardening or likely false positive.

Fourth, the LLM should improve explanation quality without controlling evidence integrity. The local LLM generates impact and minimal safe-fix text, but deterministic validation checks whether all scanner check IDs remain represented. If the LLM output drops or adds check IDs, the output is rejected.

Pipeline Components

The system contains four main scripts.

``run_scans.py`` runs Semgrep against the target applications and writes raw JSON findings into ``data/findings/``. On Windows, the current workflow uses the Semgrep Docker image so that scanning remains reproducible across local environments.

``llm_rank.py`` reads the raw Semgrep JSON files, clusters related findings, applies deterministic severity and false-positive rules, calls the local Ollama model for explanation text, and validates the final output. The output is written to ``data/rankings/``.

``eval.py`` compares raw findings, LLM-assisted clusters, and manually created labels. It reports raw finding count, cluster count, compression ratio, finding-level precision and recall, severity accuracy, false-positive triage accuracy, and missing or extra check IDs.

``run_pipeline.py`` executes the complete pipeline in order: scan, triage, and evaluate.

Evidence Preservation

Evidence preservation is the central safety property of the design. The pipeline extracts the set of Semgrep check IDs from raw scanner output and compares it with the set of check IDs present in the final triage output. The output is accepted only when the final set covers the scanner check IDs without adding unsupported check IDs.

This validation does not prove that every explanation is perfect, but it prevents a major class of LLM failure: silently dropping scanner evidence or inventing unsupported findings. The reviewer can still trace each cluster back to the original Semgrep rule, file, and line number.

Deterministic Triage Rules

The prototype currently includes deterministic rules for several vulnerability classes:

- Command injection: user-controlled input reaches subprocess execution.
- Code injection: user-controlled input reaches eval-style execution.

- SQL injection: user-controlled input reaches SQL query construction or execution.
- Subprocess hardening: subprocess usage is present, but no user-controlled input is shown reaching the command.
- Eval hardening: eval usage is present, but no user-controlled input is shown reaching the expression.

These rules are intentionally conservative. Confirmed user-controlled flows are prioritized as true positives. Dangerous API usage without demonstrated user-controlled input is retained as a lower-risk hardening case rather than discarded completely.

Local LLM Enrichment

The prototype uses a local Ollama model for enrichment. The model receives structured cluster context and is asked to generate concise impact and remediation guidance. The LLM is not allowed to define the set of findings. It does not decide which Semgrep check IDs exist, and its output is validated before being accepted.

Using a local model also supports reproducibility and privacy. Source snippets and findings do not need to be sent to an external API. This is useful for organizations that cannot share application code with third-party services.

Evaluation Design

The evaluation uses small labeled targets so that each expected result can be inspected manually. Each target includes a label file describing the expected check ID, file, line, true-positive status, severity, and category. The evaluator then compares scanner output and triage output against those labels.

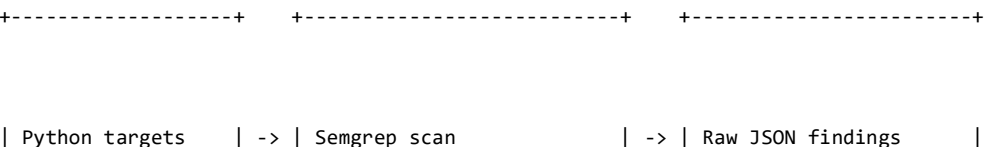
The current metrics are designed to answer three questions:

1. Does the scanner evidence remain covered after LLM-assisted triage?
2. Does clustering reduce duplicate or related findings?
3. Does the triage logic correctly distinguish confirmed vulnerabilities from hardening or false-positive cases?

The current results are early and should not be interpreted as broad generalization. They validate the architecture and measurement approach before scaling to larger datasets.

Architecture Figure

Figure 1 shows the scanner-first workflow used in this project.



(example apps)		(security-audit, owasp,		data/findings/*.json	
----------------	--	-------------------------	--	----------------------	--

		secrets rules)			
--	--	----------------	--	--	--

+-----+	+-----+	+-----+
---------	---------	---------

|

v

+-----+	+-----+	+-----+
---------	---------	---------

Manual labels	-> Evaluation	<- Deterministic triage	
---------------	-----------------	---------------------------	--

labels/*.json		precision, recall,		clustering, severity,	
---------------	--	--------------------	--	-----------------------	--

		severity, FP accuracy		evidence validation	
--	--	-----------------------	--	---------------------	--

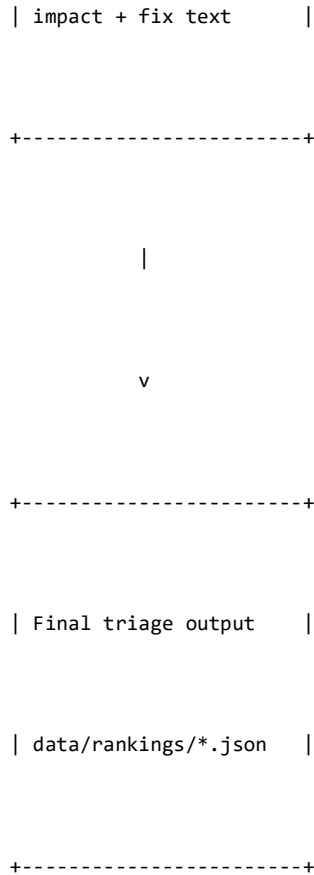
+-----+	+-----+	+-----+
---------	---------	---------

|

v

+-----+

| Local Ollama |



The key design principle is that Semgrep remains the source of truth. Deterministic logic preserves scanner evidence and performs clustering before the local LLM is used only for explanation and remediation enrichment.

Methodology

The methodology evaluates whether a scanner-first LLM-assisted workflow can make SAST findings easier to triage while preserving scanner evidence. The experiment is designed around a conservative principle: Semgrep produces the security evidence, deterministic logic organizes and validates that evidence, and the local LLM is used only to enrich the final cluster with impact and remediation text.

Experimental Setup

The prototype scans Python targets with Semgrep using Docker. Each target is a small application or code sample designed to exercise one vulnerability pattern or one hardening-only pattern. The scan output is stored as raw JSON under `data/findings/`. This raw output is treated as the source of truth for check IDs, file paths, line numbers, scanner messages, and matched code.

The LLM enrichment stage uses a local Ollama model. The model receives structured context derived from Semgrep findings and returns explanation-oriented fields such as impact, minimal safe fix, and false-positive rationale. The model is not trusted to decide which findings exist. The final output is accepted only after deterministic validation confirms that scanner check IDs have not been removed or invented.

Target Selection

The initial dataset contains five Python targets. Three targets represent confirmed vulnerability paths: subprocess command injection, eval-based code injection, and SQL injection. Two targets represent hardening or likely false-positive cases: subprocess usage without demonstrated user-controlled input and eval usage without demonstrated user-controlled input.

The targets are intentionally small. This allows each Semgrep finding, cluster, and label to be manually inspected. The goal of this stage is not to claim broad generalization, but to validate the pipeline design, evidence-preservation logic, and evaluation metrics before scaling to larger applications.

Labeling

Each target has a label file under `labels/``. A label records the expected Semgrep check ID, file path, line number, true-positive status, severity, and vulnerability category. A finding is labeled as a true positive when user-controlled input reaches a dangerous operation. A finding is labeled as false positive or hardening-only when the scanner reports a risky API or construct but the available code does not show user-controlled input reaching the operation.

Severity labels are assigned based on exploitability in the small target. Confirmed command injection, code injection, and SQL injection cases are labeled High. Hardening-only eval and subprocess cases are labeled Low.

Evaluation Metrics

The evaluator reports raw Semgrep findings, LLM triage clusters, cluster compression ratio, finding-level precision, finding-level recall, severity accuracy, false-positive triage accuracy, missing check IDs, and extra check IDs.

The compression ratio measures how many raw findings are represented by each triage cluster. Precision and recall are computed against the manual labels. Severity accuracy checks whether the cluster severity matches the expected label severity. False-positive triage accuracy checks whether the pipeline correctly marks true vulnerabilities as not false positives and hardening-only cases as likely false positives. Missing and extra check ID counts measure evidence preservation.

Reproducibility

The full experiment can be reproduced by running `python scripts/run_pipeline.py``. This executes scanning, triage, and evaluation in order. The repository also includes a results snapshot, evaluation table, experiment manifest, and reproducibility guide so that the reported baseline can be inspected without relying only on generated local artifacts.

Results

This section summarizes the current prototype evaluation.

Dataset

The current evaluation uses five small Python targets:

Target	Purpose
`myapp`	Real subprocess command injection
`evalapp`	Real eval code injection
`sqlapp`	Real SQL injection
`safeapp`	Subprocess hardening / false-positive case
`safeevalapp`	Eval hardening / false-positive case

Overall Results

Metric	Value
Targets evaluated	5
Total raw Semgrep findings	9
Total LLM triage clusters	5
Overall compression ratio	1.80x
Overall precision	0.78
Overall recall	1.00
Overall severity accuracy	1.00
Overall false-positive triage accuracy	1.00
Missing Semgrep check IDs	0
Extra Semgrep check IDs	0

Interpretation

The prototype reduced nine raw Semgrep findings into five triage clusters while preserving all scanner check IDs. The initial results show that deterministic clustering can reduce duplicate or related findings without losing scanner evidence.

The precision value is lower than 1.00 because the dataset intentionally includes false-positive or hardening-only findings. This is expected. The more important measurement for those cases is false-positive triage accuracy, which is currently 1.00 on the initial dataset.

The current result should be treated as an early validation of the pipeline design, not as a general claim about all SAST findings or all application frameworks.

Evaluation Table

Table 1 summarizes the initial five-target evaluation.

Target	Raw Findings	LLM Clusters	Compression	TP Labels	FP Labels	Precision	Recall	Severity Accuracy	FP Triage Accuracy
myapp	3	1	3.00x	3	0	1.00	1.00	1.00	1.00
evalapp	3	1	3.00x	3	0	1.00	1.00	1.00	1.00

sqlapp	1	1	1.00x	1	0	1.00	1.00	1.00	1.00
safeapp	1	1	1.00x	0	1	0.00	0.00	1.00	1.00
safeevalapp	1	1	1.00x	0	1	0.00	0.00	1.00	1.00
Overall	9	5	1.80x	7	2	0.78	1.00	1.00	1.00

The results show that the pipeline reduced nine raw findings into five clusters while preserving scanner evidence. The overall precision is lower than recall because the dataset intentionally includes hardening-only and likely false-positive cases, but severity accuracy and false-positive triage accuracy remained perfect in this initial evaluation.

Limitations and Threats to Validity

This prototype is an initial research artifact and should not be interpreted as a production-ready SAST triage system.

Dataset Size

The initial evaluation uses a small set of python examples. This is sufficient for prototyping the pipeline and measurements, but it is not large enough to claim general effectiveness across all languages, frameworks, or vulnerability classes.

Synthetic Targets

The current targets are small and controlled. Real-World applications may include more complex data flows, framework abstractions, indirect function calls, and dependency-driven behavior that may change scanner and triage results.

Scanner Dependency

The pipeline depends on Semgrep check IDs, finding format, and rule behavior. Different Semgrep versions, proprietary rule packs, or custom rules could produce different findings.

LLM Variability

The project uses a local Ollama model for enrichment. Different models may generate different impact and fix text. The prototype mitigates this risk by preserving SAST check IDs deterministically and validating that the LLM does not drop invariant evidence.

Label Bias

Labels are manually created for this initial dataset. False-positive labels and severity labels may require additional internal review or external benchmarking before publication.

Next Validation Steps

- Add more applications and frameworks.
- Add more false-positive hardening cases.
- Report per-class results.
- Compare against raw Semgrep severity and a baseline manual triage workflow.
- Test different local LLM models for stability and output consistency.

Conclusion and Future Work

This project presents an initial scanner-first approach for LLM-assisted SAST triage. The prototype keeps Semgrep as the source of truth, applies deterministic clustering and rule-based triage before LLM enrichment, and validates that scanner check IDs are preserved in the final output. This design addresses a central risk in LLM-assisted security workflows: the possibility that generated explanations may omit, invent, or alter evidence.

The initial evaluation shows that the prototype can reduce related SAST findings into smaller triage clusters while preserving scanner evidence. Across five Python targets, the system reduced nine raw Semgrep findings to five triage clusters, preserved all scanner check IDs, and correctly distinguished confirmed vulnerability paths from hardening or likely false-positive cases in the labeled dataset. These results are early and limited, but they demonstrate that controlled LLM assistance can improve readability without giving the LLM authority over the evidence trail.

The main lesson is that LLMs are most useful in this workflow when they are constrained. The LLM should not decide which findings exist, which check IDs matter, or whether scanner evidence should be removed. Instead, deterministic logic should preserve evidence and make high-confidence triage decisions, while the LLM improves communication through impact summaries and remediation guidance.

Future work should expand the evaluation in several directions. First, the dataset should include more applications, larger codebases, and additional vulnerability classes such as path traversal, SSRF, insecure deserialization, insecure cryptography, and hardcoded secrets. Second, the project should compare multiple local LLMs to measure stability of impact and remediation text. Third, the evaluation should report per-class metrics so that command injection, SQL injection, code injection, and hardening-only cases can be analyzed separately. Fourth, the prototype should be tested against real open-source projects and benchmark datasets such as SAST triage benchmarks.

A longer-term direction is to integrate this workflow into developer review systems, where SAST findings are grouped into evidence-preserving clusters before being shown in pull requests or security dashboards. In that setting, the core safety principle should remain unchanged: scanner evidence is authoritative, deterministic validation is mandatory, and the LLM is used only to make validated findings easier to understand and fix.